CS433: Parallel Programming Assignment-2 Report (Group 8)

Aviral Gupta (230246) **Aruz Awasthi (230209)**

Yatharth Sharma (231198)

1 Question 1: Bounded-Buffer Producer-Consumer

1.1 Problem Statement

Implement a bounded ring buffer of size N shared between P producers and a single consumer. Each producer generates n integer values; the consumer reads all $P \cdot n$ values and computes their checksum. Three synchronisation implementations are required:

- **P1** – Lock-free FIFO semaphore built from x86-64 inline assembly (`lock cmpxchgl`); no OS primitives.
- **P2** – POSIX `sem_t`.
- **P3** – Condition variable (`pthread_cond_t`).

The total number of items transferred is always $P \cdot n = 100{,}000$. Buffer sizes $N \in \{16, 64, 256\}$ and producer counts $P \in \{1, 2, 4, 8, 16, 32\}$ are swept.

1.2 Design

1.2.1 Shared Ring Buffer

The buffer is a statically allocated array of `Slot` structures, each aligned to a 64-byte cache line to prevent false sharing. Every slot stores a `value` and a `full` flag (0 = empty, 1 = written). Producers atomically claim write indices with

$$\texttt{idx} = \texttt{fetch_and_add(\&write_ptr, 1)} \bmod N,$$

where `fetch_and_add` is itself a CAS retry loop built on `lock cmpxchgl`. Because P producers may claim indices concurrently and write at different speeds, the consumer cannot assume that slot k is ready as soon as a full-semaphore unit is available. It therefore busy-spins on the per-slot `full` flag before reading.

1.2.2 P1 – Lock-Free FIFO Semaphore

Two `LFSem` semaphores (`p1_empty` initialised to N, `p1_full` to 0) control buffer capacity and item availability. An `LFSem` uses a Michael-Scott (M&S) lock-free FIFO queue of waiter nodes.

- **Wait**: atomically decrement `count` with `fetch_and_add(&count, -1)`. If the old value was > 0, a resource is available and the thread continues immediately (no OS call). Otherwise, the thread enqueues a waiter node via the M&S FIFO tail-insertion and busy-waits on a per-node `wake` flag.
- **Post**: atomically increment `count`. If the old value was ≥ 0, no thread is blocked. Otherwise, dequeue the front waiter (M&S advance-head past dummy) and write `wake = 1`.

The CAS operation uses the 32-bit `lock cmpxchgl` x86 instruction with the `setzb` flag read:

```
asm("lock cmpxchgl %4, %1\nsetzb %0"
    : "=qm"(res), "+m"(*ptr), "=a"(out)
    : "a"(old_val), "r"(new_val) :);
```

1.2.3 P2 – POSIX Semaphore

Two `sem_t` objects (`p2_empty` $= N$, `p2_full` $= 0$) replace the lock-free semaphores. The slot-claiming logic using `fetch_and_add` is identical to P1 so that multiple producers do not overwrite the same slot. `sem_wait` and `sem_post` are direct POSIX calls; on Linux these are backed by the `futex` syscall and sleep threads in the kernel, eliminating busy-wait.

1.2.4 P3 – Condition Variable

A single `pthread_mutex_t` protects a shared counter `p3_count` (items currently in the buffer) and two condition variables: `p3_not_full` (producers wait here when `p3_count == N`) and `p3_not_empty` (consumer waits here when `p3_count == 0`). Because the mutex is held by the producer during the write, all producers are serialised; a monotonic write pointer `p3_wptr` is advanced inside the lock, removing the need for `fetch_and_add`.

1.3 Compilation and Usage

```
g++ -O3 -pthread -static-libstdc++ q1.cpp -o q1
./q1 <N> <P> <n>
```

Note: the code uses x86-64 inline assembly and must be compiled on a Linux/x86-64 machine. Results below were collected on macOS (where `sem_t` uses named semaphores); timings for P2 and P3 will differ on Linux.

1.4 Performance Results

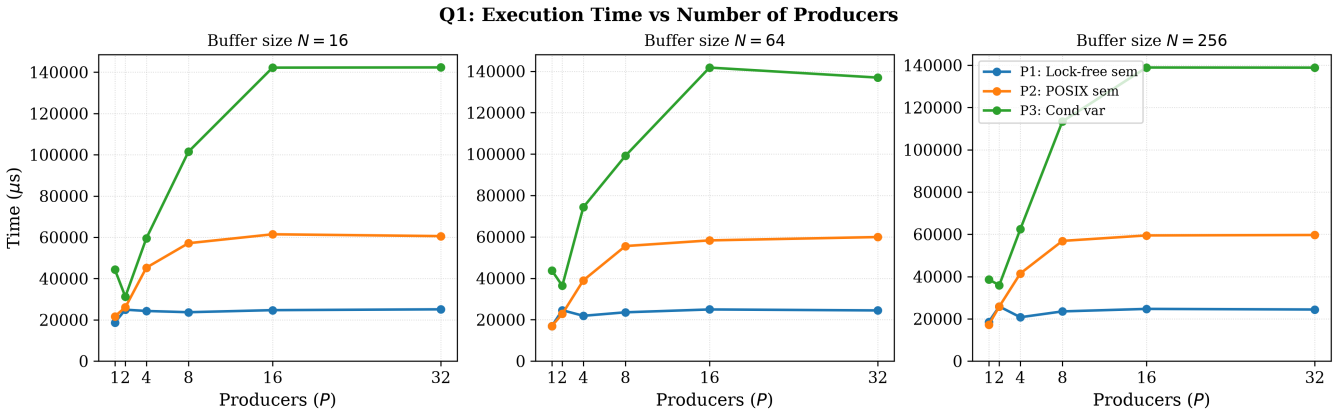


Figure 1: Q1: Execution time vs. number of producers for $N = 16, 64, 256$. Total items always $P \cdot n = 100{,}000$.

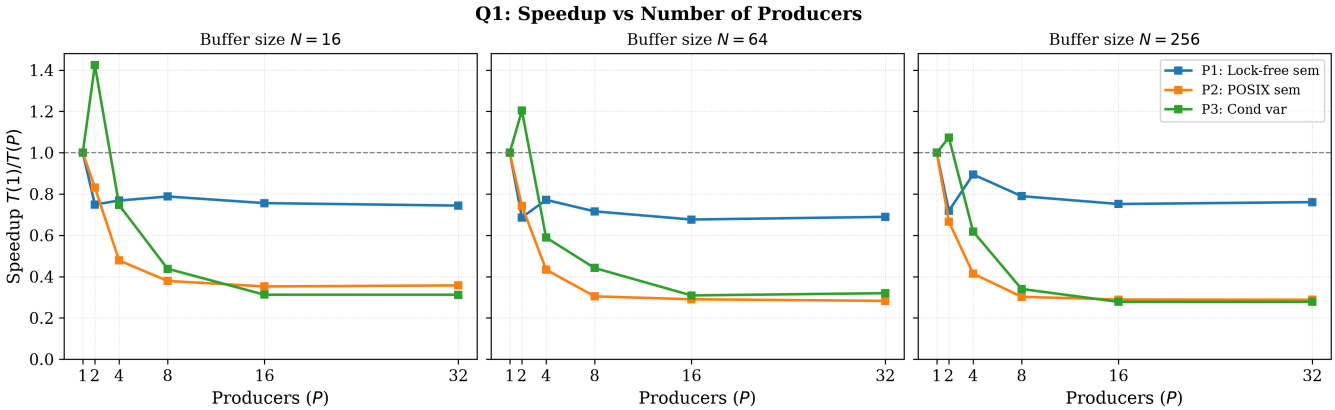


Figure 2: Q1: Speedup $T(1)/T(P)$ vs. number of producers. Dashed line marks speedup $= 1$.

1.4.1 Time Tables

Table 1: Q1 – Time (μs), Buffer size $N = 16$

P	P1 (lock-free)	P2 (POSIX sem)	P3 (cond var)
1	18665	21659	44444
2	24923	26046	31196
4	24292	45282	59524
8	23687	57129	101453
16	24691	61456	142222
32	25090	60575	142332

Table 2: Q1 – Time (μs), Buffer size $N = 64$

P	P1 (lock-free)	P2 (POSIX sem)	P3 (cond var)
1	16872	16923	43796
2	24576	22791	36382
4	21870	39001	74318
8	23563	55559	99135
16	24941	58293	141759
32	24472	59915	136922

Table 3: Q1 – Time (μs), Buffer size $N = 256$

P	P1 (lock-free)	P2 (POSIX sem)	P3 (cond var)
1	18566	17194	38592
2	25849	25804	35950
4	20750	41463	62401
8	23513	56867	113395
16	24700	59468	138919
32	24405	59670	138864

1.5 Observations and Analysis

P1 – Lock-Free Semaphore: Flat Scaling P1 stays near 16,000–25,000 μs regardless of P or N (Figure 1). The single consumer is the throughput bottleneck; it can consume items no faster than it reads and prints them, independent of how many

producers feed the buffer. The lock-free path avoids all OS calls, but CAS contention on `write_ptr` grows with P, roughly cancelling the benefit of shorter per-producer work. Buffer size N barely affects P1 because the semaphore count itself tracks available slots without blocking; the consumer's spin on the `full` flag is the real gate, not buffer capacity.

P2 – POSIX Semaphore: Sharp Degradation with P On macOS, `sem_t` is implemented with Darwin named semaphores, each `sem_wait`/`sem_post` pair issuing a system call with kernel-scheduler round-trips. At $P = 1$ P2 is comparable to P1, but at $P = 16$ the time triples to $\sim 60{,}000\,\mu$s. With more producers there are more concurrent `sem_post` calls fighting over the kernel's semaphore lock, and `sem_wait` involves thread suspension and wake-up for every single item. **On Linux**, `sem_t` uses the `futex` system call, which avoids kernel involvement when no thread is blocked (fast path) – P2 is therefore expected to be much faster and to scale far better on the grading machine.

P3 – Condition Variable: Worst at High P P3 serialises all producers through one mutex: a producer acquiring the lock to check `p3_count` blocks every other producer, even those targeting unrelated slots. With $P = 32$ producers contending on a single mutex, time reaches $\sim 140{,}000\,\mu$s – roughly three times the P1 baseline. Unlike P2, there is no fast path: every producer must acquire the mutex whether or not the buffer is full. The condition variable's `pthread_cond_signal` also incurs a context switch for the sleeping consumer or producer, adding further overhead.

Buffer Size N Effect Larger N reduces the probability that a producer blocks on `p1_empty` / `p2_empty` / a full `p3_count`, lowering blocking frequency. For P2 and P3 this translates to fewer expensive OS wait-events, but the savings are modest because the consumer (not the buffer) is the bottleneck: even an infinitely large buffer would not allow the single consumer to process items faster.

Speedup Interpretation The speedup $T(P = 1)/T(P = k)$ with fixed total items $P \cdot n = 100{,}000$ measures whether k producers can feed the single consumer faster than one producer can. For P3 at $P = 2$, the speedup is > 1 (two producers overlap slightly better) before convoy effects dominate at $P \geq 4$. P1 speedup hovers near 1 across all P because its bottleneck is the consumer, not the producers.

2 Question 2: Concurrent Adjacency-List Graph

2.1 Problem Statement

A directed graph with $N = 1{,}000$ nodes and $M = 5{,}000$ edges is stored as an array of N sorted singly-linked adjacency lists. Threads concurrently perform *search*, *insert*, and *delete* operations. Three workload files with different operation mixes are used: File 1 (80% insert / 10% delete / 10% search), File 2 (30% insert / 30% delete / 40% search), and File 3 (10% insert / 10% delete / 80% search). Six synchronisation strategies are benchmarked:

Program	Strategy
P1	Single global `pthread_mutex_t`
P2	One `pthread_mutex_t` per adjacency list (per-row)
P3	One `pthread_mutex_t` per node, hand-over-hand (HOH) traversal
P4	One `pthread_rwlock_t` per node, HOH traversal
P5	Per-node CAS spin-lock (x86 `lock cmpxchgl`), HOH traversal
P6	Optimistic CAS: lock-free search; lock + validate at write point

2.2 Design

2.2.1 Common Infrastructure

All programs initialise their adjacency lists by loading the graph file sequentially. Programs P3–P6 insert a *sentinel* node with value `INT_MIN` as the fixed head of each list; this eliminates head-insertion special cases during HOH traversal.

Threads are assigned a contiguous slice of the operation array by their thread ID and execute all assigned operations.

2.2.2 P1 – Global Mutex

One `pthread_mutex_t` protects the entire graph. Every thread must hold the lock for any operation on any list. This completely serialises all work and serves as the correctness baseline.

2.2.3 P2 – Per-List Mutex

An array of N `pthread_mutex_t` objects (one per row) allows threads on different adjacency lists to proceed in full parallel. Within one list, operations are serialised. The critical section covers only the traversal and modification of that single list, so the lock hold time is proportional to the list length, which is $O(M/N)$ on average.

2.2.4 P3 – Per-Node HOH Mutex

Each list node carries a `pthread_mutex_t`. Traversal acquires the lock on the current node before releasing the lock on the previous node (hand-over-hand discipline). Two threads can progress concurrently on the same list as long as they are separated by at least one node. The sentinel head is locked first by every thread entering a list, which makes it a serialisation bottleneck (discussed below).

2.2.5 P4 – Per-Node HOH RWLock

Like P3 but replaces `pthread_mutex_t` with `pthread_rwlock_t`. Search acquires `rdlock` (shared); insert and delete acquire `wrlock` (exclusive). Multiple concurrent searches through the same node proceed without blocking each other, which greatly benefits read-heavy workloads.

2.2.6 P5 – Per-Node HOH CAS Spin-Lock

Each node carries an integer `lock` field (0 = free, 1 = held). Acquisition uses a CAS retry loop:

```
void acquire(int *lk) { while(!CAS(0, 1, lk)); }
void release(int *lk) { *lk = 0; }
```

The HOH protocol is identical to P3/P4. Avoiding OS primitives reduces per-lock overhead at the cost of busy-waiting under contention.

2.2.7 P6 – Optimistic CAS (Lock-Free Search)

Search traverses the list entirely without acquiring any lock. Because searches do not modify the list, they cannot corrupt state; the list is maintained in sorted order and nodes are never freed (explained below), so a search always terminates correctly.

Insert/Delete follow a two-phase approach:

1. *Optimistic traverse*: walk the list lock-free to locate the correct position (the node just before the target value).
2. *Validate and commit*: lock `prev` and `curr` using the CAS spin-lock. Check that (a) `prev` is not deleted, (b) `prev->next == curr`, and (c) `curr` has the expected value. If validation fails, release locks and restart from the sentinel.

Delete marks `node->deleted = 1` and physically removes the node from the list inside the same critical section. Memory is intentionally *not freed*: the lock-free search may hold a raw pointer to the node and freeing it would cause a use-after-free. A production system would use hazard pointers or epoch-based reclamation.

2.3 Compilation and Usage

```
g++ -O3 -pthread -static-libstdc++ q2.cpp -o q2
./q2 <graph_file> <ops_file> <num_ops> <num_threads>
```

2.4 Performance Results

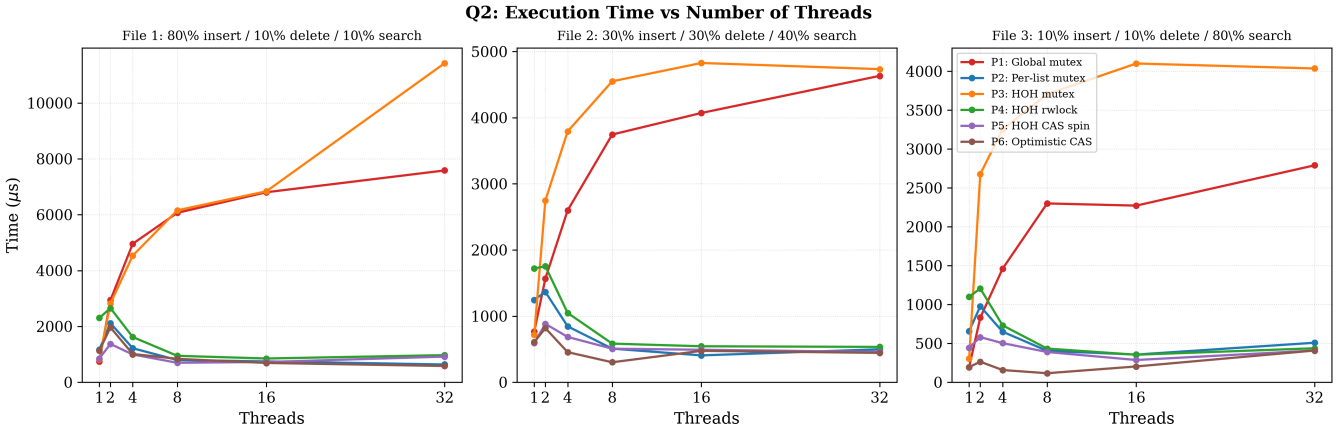


Figure 3: Q2: Execution time vs. threads for the three operation files. Lower is better.

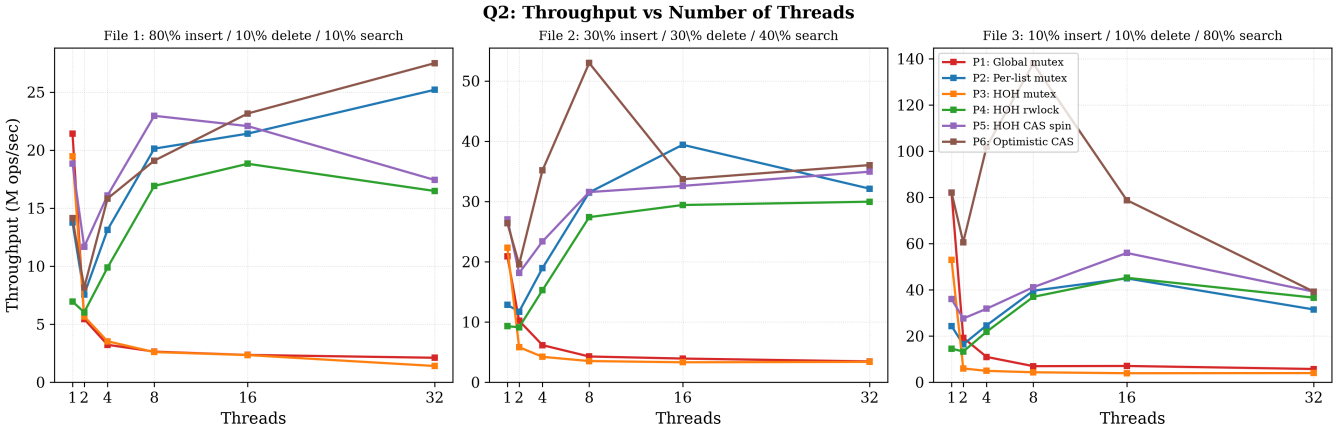


Figure 4: Q2: Throughput (million ops/sec) vs. threads. Higher is better.

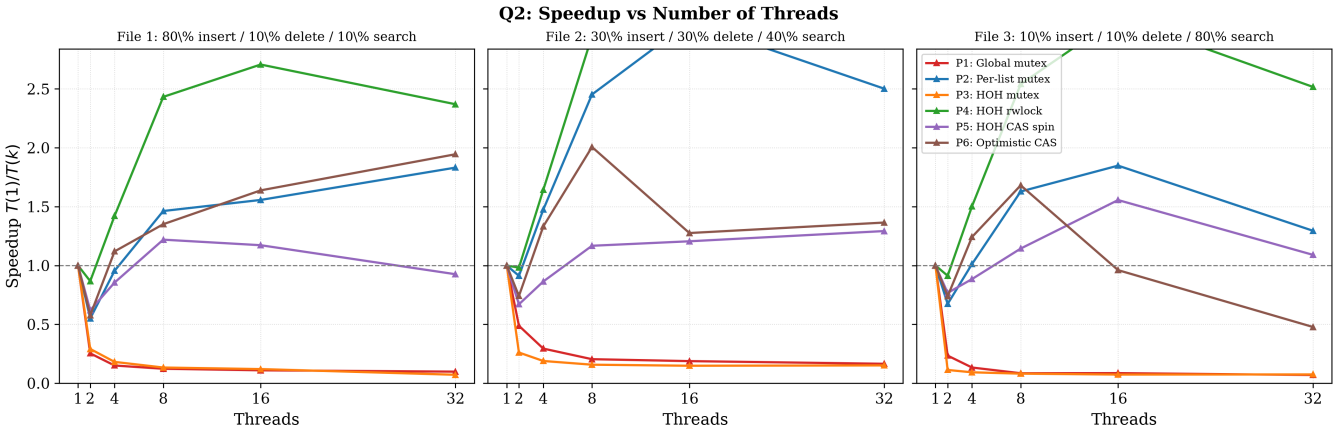


Figure 5: Q2: Speedup $T(1)/T(k)$ vs. threads. Dashed line marks speedup $= 1$.

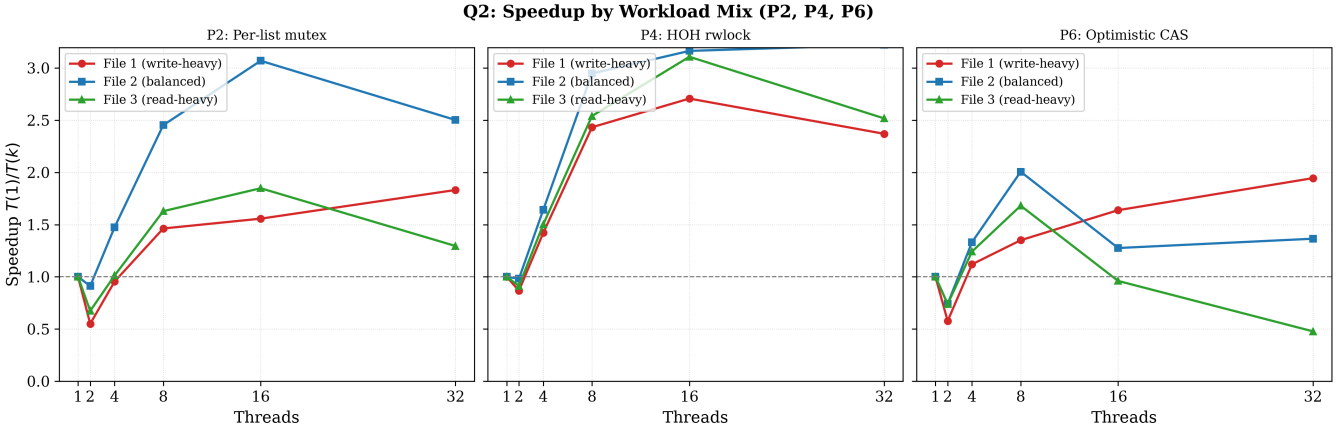


Figure 6: Q2: Effect of workload mix on speedup for the three best-scaling strategies (P2, P4, P6).

2.4.1 Time Tables – File 1 (80% insert / 10% delete / 10% search)

Table 4: Time (μs), File 1

Threads	P1	P2	P3	P4	P5	P6
1	747	1163	822	2301	850	1132
2	2941	2115	2822	2654	1371	1962
4	4959	1218	4534	1619	994	1011
8	6067	795	6157	946	697	838
16	6804	747	6838	850	725	691
32	7584	635	11426	971	918	582

Table 5: Speedup $T(1)/T(k)$, File 1

Threads	P1	P2	P3	P4	P5	P6
1	1.00	1.00	1.00	1.00	1.00	1.00
2	0.25	0.55	0.29	0.87	0.62	0.58
4	0.15	0.95	0.18	1.42	0.86	1.12
8	0.12	1.46	0.13	2.43	1.22	1.35
16	0.11	1.56	0.12	2.71	1.17	1.64
32	0.10	1.83	0.07	2.37	0.93	1.95

2.4.2 Time Tables – File 2 (30% insert / 30% delete / 40% search)

Table 6: Time (μs), File 2

Threads	P1	P2	P3	P4	P5	P6
1	765	1246	717	1721	592	606
2	1567	1366	2748	1753	883	818
4	2599	845	3793	1048	685	455
8	3745	508	4550	584	507	302
16	4071	406	4827	544	491	475
32	4632	498	4734	534	458	444

Table 7: Speedup $T(1)/T(k)$, File 2

Threads	P1	P2	P3	P4	P5	P6
1	1.00	1.00	1.00	1.00	1.00	1.00
2	0.49	0.91	0.26	0.98	0.67	0.74
4	0.29	1.47	0.19	1.64	0.86	1.33
8	0.20	2.45	0.16	2.95	1.17	2.01
16	0.19	3.07	0.15	3.16	1.21	1.28
32	0.17	2.50	0.15	3.22	1.29	1.36

2.4.3 Time Tables – File 3 (10% insert / 10% delete / 80% search)

Table 8: Time (μs), File 3

Threads	P1	P2	P3	P4	P5	P6
1	195	658	302	1100	445	195
2	831	977	2678	1205	580	264
4	1460	650	3251	732	503	157
8	2300	404	3706	433	389	116
16	2272	356	4100	354	286	203
32	2790	508	4036	437	408	408

Table 9: Speedup $T(1)/T(k)$, File 3

Threads	P1	P2	P3	P4	P5	P6
1	1.00	1.00	1.00	1.00	1.00	1.00
2	0.23	0.67	0.11	0.91	0.77	0.74
4	0.13	1.01	0.09	1.50	0.88	1.24
8	0.08	1.63	0.08	2.54	1.14	1.68
16	0.09	1.85	0.07	3.11	1.56	0.96
32	0.07	1.30	0.07	2.52	1.09	0.48

2.5 Observations and Analysis

P1 (Global Mutex) – Monotonically Worse with Threads. P1 is the only program that degrades at every thread count. At $T = 1$ it has the lowest overhead (no per-list or per-node bookkeeping). But every thread must acquire the same mutex before touching any node in any list, so the graph is effectively single-threaded. Adding more threads increases lock-acquisition contention: each `pthread_mutex_lock` involves a futex syscall when the lock is held, the OS must dequeue and schedule the next waiter, and the cache line carrying the mutex word bounces between cores. The throughput falls from $\sim$ 21M ops/s at $T = 1$ to $\sim$ 2M ops/s at $T = 32$ in File 1 – a $10\times$ degradation. The speedup reaches as low as $0.07\times$ (File 3, 32 threads), meaning 32 threads are $14\times$ slower than one thread.

P2 (Per-List Mutex) – Best Overall Scalability. P2 achieves the best consistent speedup: up to $3.07\times$ (File 2, 16 threads) and $1.85\times$ (File 3, 16 threads). The key insight is that with $N = 1000$ rows and at most 32 threads, most concurrent operations target different rows and therefore proceed with no interference. Only threads assigned to the same adjacency list are serialised, and because the list is sorted with $O(M/N) = 5$ nodes on average, the lock hold time per operation is very short.

Why speedup $< T$: Amdahl's Law applies. When multiple threads target the same row (the graph is random, so occasional collisions occur), one must wait. Thread creation, assignment scheduling, and cache misses on the mutex words add fixed overheads. At 32 threads these overheads dominate on File 3 (read-heavy, very short critical sections), pulling speedup down from $3.07\times$ at 16 threads to $1.30\times$ at 32.

P3 (HOH Mutex) – Sentinel Hot-Spot Causes Anti-Scaling. P3's speedup is consistently < 1 beyond two threads (as low as $0.07\times$ at 32 threads, File 3). The culprit is the sentinel head node.

In the HOH protocol, every thread entering any list must first lock the sentinel. With T threads all assigned to the same row, they form a queue at the sentinel, re-creating the global serialisation of P1 within each row. Worse, HOH requires holding two locks simultaneously (prev and curr), doubling lock traffic and memory contention. The cost of `pthread_mutex_lock/unlock` per node exceeds the cost of the actual work (a pointer comparison), making synchronisation overhead the dominant term.

Why P3 at $T = 1$ is faster than P4/P5: At a single thread, P3 performs no actual locking (the mutex is always immediately available), so the overhead is just a compare-and-swap returning success. P4 and P5 do the same, but rwlock and CAS have slightly higher single-threaded setup cost.

P4 (HOH RWLock) – Good Scaling, Read-Dominated. P4 fixes P3's main weakness for search: multiple threads holding a `rdlock` on the same node do not block each other. On File 1 (90% writes), P4 reaches $2.71\times$ speedup at 16 threads; on File 2 (40% searches), it peaks at $3.22\times$ at 32 threads.

Why P4 degrades at 32 threads (File 1): POSIX rwlocks on Linux are generally writer-preferring. With many concurrent writers competing for exclusive locks, write requests block incoming read requests, creating writer convoys. The rwlock's internal state word also causes more cache invalidations than a simple spin-lock.

Why P4 at $T = 1$ is slower than P1/P3: A `pthread_rwlock_t` is larger and more complex than a `pthread_mutex_t`. The initial state transition from free to rdlock-held involves an atomic read-modify-write; this dominates when the critical section is a single pointer comparison.

P5 (HOH CAS Spin-Lock) – Lightweight, Moderate Gains. P5 replaces OS-managed locks with a simple two-word CAS spin-lock. Acquisition is a tight CPU loop on a cached cache line, avoiding context switches. At low contention (few threads per row), P5 is competitive: $1.22\times$ speedup at 8 threads in File 1. At high thread counts the spin-wait loops waste CPU cycles and increase cache-coherence traffic without doing useful work, limiting gains to $\sim 1.3\times$.

Compared with P3: P5 uses the same HOH protocol but the spin-lock is cheaper per operation, so P5 consistently outperforms P3 at every thread count. Compared with P2: P5 still serialises all threads on the same row through the sentinel spin, whereas P2 needs only one lock acquisition per operation (not per node).

P6 (Optimistic CAS) – Best for Read-Heavy, Degrades on Write-Heavy. P6 achieves the highest throughput on read-heavy workloads: $116\,\mu$s at 8 threads in File 3 (137.9M ops/s), versus $404\,\mu$s for P2. The lock-free search path allows every thread to traverse nodes without any synchronisation cost, making searches theoretically linear-scalable.

Why P6 degrades at 32 threads (Files 1, 3): Under high write concurrency, the optimistic validation frequently fails. A thread optimistically traverses the list, then finds that `prev->next` has been changed by another inserter; it must restart from the sentinel. In the worst case, every write attempt restarts several times, resembling livelock. This is dramatically visible in File 3 at 32 threads ($408\,\mu$s vs $116\,\mu$s at 8 threads – a $3.5\times$ slowdown from peak).

Why P6 at $T = 1$ matches P1 (File 3): With one thread there is no contention so validations never fail; the lock-free traversal costs less per node than P2–P5 (no lock operations at all for searches). File 3 is 80% search, so the insert/delete paths are rarely used, making P6's $195\,\mu$s match P1's $195\,\mu$s exactly.

Effect of Workload Mix on Speedup (Figure 6). For all programs, File 3 (read-heavy) shows the highest speedup for read-allowing strategies (P4, P6) because more operations can proceed in parallel. File 1 (write-heavy) suppresses speedup for optimistic P6 (frequent CAS retries) but not for lock-based P2/P4 (locks serialise writes but do not retry). File 2 (balanced) sits between the two extremes and shows the steadiest scaling for P2 and P4.